Edge cases (an unhandled routing value) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `conditional_routing_practice.py` (Edge cases section): an unmapped routing value has to fail loudly — this exercise proves it does, on purpose, before you ever see it happen by accident in a real graph. **If not:** the first unmapped routing value in a real graph would be a mystery `KeyError` several stack frames deep inside LangGraph, with no smaller example telling you what it means.

Read both depths — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — trigger it and read what happens

```
# conditional_routing_practice.py — Edge cases section
from typing import TypedDict
from langgraph.graph import StateGraph, START, END


class GraphState(TypedDict):
    flag: bool | None
    message: str


def node_decide(state):
    return {}


def node_a(state):
    return {"message": "path A"}


def node_b(state):
    return {"message": "path B"}
```

```
def route(state):
    if state["flag"] is None:
        return "path_c"  # deliberately not a key in the mapping below
    if state["flag"]:
        return "path_a"
    return "path_b"


builder = StateGraph(GraphState)
builder.add_node("node_decide", node_decide)
builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
builder.add_edge(START, "node_decide")
builder.add_conditional_edges(
    "node_decide", route, {"path_a": "node_a", "path_b": "node_b"},
)
builder.add_edge("node_a", END)
builder.add_edge("node_b", END)

graph = builder.compile()
print("compiled fine")

print(graph.invoke({"flag": True, "message": ""}))

try:
    graph.invoke({"flag": None, "message": ""})
except Exception as e:
    print(f"{type(e).__name__}: {e}")
```

Expected output:

```
compiled fine
{'flag': True, 'message': 'path A'}
KeyError: 'path_c'
```

Two things to notice: `builder.compile()` never raises anything — LangGraph doesn't check that a routing function's every possible return value is covered by the mapping at compile time, because it can't know every value a Python function might return without running it. The normal case (`flag=True`) still works exactly as before. Only the actual bad case — `flag=None`, which makes `route` return `"path_c"` — raises, and it raises loudly with a `KeyError` naming the unmapped value, not a silent wrong path.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — confirming exactly when it fails, with type hints

```
# conditional_routing_practice.py — Edge cases section
from typing import TypedDict
from langgraph.graph import StateGraph, START, END


class GraphState(TypedDict):
    flag: bool | None
    message: str


def node_decide(state: GraphState) -> dict:
    return {}


def node_a(state: GraphState) -> dict:
    return {"message": "path A"}


def node_b(state: GraphState) -> dict:
    return {"message": "path B"}


def route(state: GraphState) -> str:
    if state["flag"] is None:
        return "path_c"
    if state["flag"]:
        return "path_a"
    return "path_b"


builder = StateGraph(GraphState)
builder.add_node("node_decide", node_decide)
builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
builder.add_edge(START, "node_decide")
builder.add_conditional_edges(
    "node_decide", route, {"path_a": "node_a", "path_b": "node_b"},
```

```
)
builder.add_edge("node_a", END)
builder.add_edge("node_b", END)

# why: compiling never runs route() -- it only registers graph structure
graph = builder.compile()

test_cases = [True, False, None]
for flag_value in test_cases:
    try:
        result = graph.invoke({"flag": flag_value, "message": ""})
        print(f"flag={flag_value}: OK -> {result['message']}")
    except KeyError as e:
        print(f"flag={flag_value}: FAILED LOUDLY -> KeyError({e})")
```

Expected output:

```
flag=True: OK -> path A
flag=False: OK -> path B
flag=None: FAILED LOUDLY -> KeyError('path_c')
```

Difference from Basic: same finding, with the check run across all 3 input cases in a loop instead of one manual `try` / `except`, and the `KeyError` caught specifically (instead of the broad `Exception`) now that Basic already confirmed that's the actual type raised — narrowing the `except` clause to the real exception type is the more correct pattern once you know what you're actually catching.

Approach 2 — a self-checking routing function with a clear custom exception

Story: the library's own `KeyError` correctly fails loudly, but its message doesn't say anything about *your* routing function or *why* the value was wrong — just that a key lookup failed somewhere inside LangGraph's branch-running code. **If not:** debugging a real routing bug at 2am would mean tracing a generic `KeyError` back through several stack frames of library code instead of reading one message that already names the cause.

```
# conditional_routing_practice.py — Edge cases section
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

VALID_PATHS = {"path_a", "path_b"}


class UnhandledRouteError(Exception):
    """Raised when route() produces a value the graph's mapping
    doesn't handle."""
```

```
class GraphState(TypedDict):
    flag: bool | None
    message: str


def node_decide(state: GraphState) -> dict:
    return {}


def node_a(state: GraphState) -> dict:
    return {"message": "path A"}


def node_b(state: GraphState) -> dict:
    return {"message": "path B"}


def route(state: GraphState) -> str:
    if state["flag"] is None:
        result = "path_c"
    else:
        if state["flag"]:
            result = "path_a"
        else:
            result = "path_b"

    if result not in VALID_PATHS:
        raise UnhandledRouteError(
            f"route() returned {result!r} for state {state!r}, "
            f"which isn't one of the handled paths: {VALID_PATHS}"
        )
    return result


builder = StateGraph(GraphState)
builder.add_node("node_decide", node_decide)
builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
builder.add_edge(START, "node_decide")
builder.add_conditional_edges(
    "node_decide", route, {"path_a": "node_a", "path_b": "node_b"},
)
builder.add_edge("node_a", END)
builder.add_edge("node_b", END)
```

```
graph = builder.compile()

try:
    graph.invoke({"flag": None, "message": ""})
except UnhandledRouteError as e:
    print(f"UnhandledRouteError: {e}")
```

Expected output:

```
UnhandledRouteError: route() returned 'path_c' for state
{'flag': None, 'message': ''}, which isn't one of the handled
paths: {'path_a', 'path_b'}
```

(shown wrapped onto three lines just to fit the page — really one line of output)

Compare this to Approach 1's `KeyError: 'path_c'` — both fail immediately, at the same point in execution. This one uses a specific, named exception class (not a generic `ValueError`) whose message names the actual routing function, shows the full state that caused the problem, and lists every path that *would* have worked — everything you'd want to know to fix it, in one line, and a type you can catch specifically without also catching unrelated value errors elsewhere in the same `try` block.

Approach 3 — a fallback path instead of a hard failure, used deliberately

Story: sometimes failing loudly isn't actually what you want in production — you might prefer routing anything unexpected to a dedicated "unhandled" node that logs the problem and gives a safe response, rather than crashing the whole run. **If not:** the only 2 choices left would be "crash the whole run" or "silently do the wrong thing" — a deliberate fallback node is the real third option, chosen on purpose, not fallen into by accident.

Add this below Approach 2's code in the same file — it reuses `GraphState`, `node_decide`, `node_a` and `node_b` from there.

```
# conditional_routing_practice.py — Edge cases section
import logging

logger = logging.getLogger(__name__)


def route_with_fallback(state: GraphState) -> str:
    if state["flag"] is None:
        return "unhandled"
    if state["flag"]:
        return "path_a"
    return "path_b"
```

```
def unhandled_node(state: GraphState) -> dict:
    logger.warning(f"Unhandled routing case for state: {state}")
    return {"message": f"Unhandled routing case for state: {state}"}


builder = StateGraph(GraphState)
builder.add_node("node_decide", node_decide)
builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
builder.add_node("unhandled", unhandled_node)
builder.add_edge(START, "node_decide")
builder.add_conditional_edges(
    "node_decide",
    route_with_fallback,
    {"path_a": "node_a", "path_b": "node_b", "unhandled": "unhandled"},
)
builder.add_edge("node_a", END)
builder.add_edge("node_b", END)
builder.add_edge("unhandled", END)

graph = builder.compile()
result = graph.invoke({"flag": None, "message": ""})
print(result["message"])
```

Expected output (the `print` line; the `logger.warning` line lands on stderr via your logging config, not shown here):

```
Unhandled routing case for state: {'flag': None, 'message': ''}
```

Notice this isn't "silently doing something unexpected" the way the exercise warns against — `"unhandled"` is a real, named node in the mapping, chosen on purpose, that visibly records what happened. `unhandled_node` calls `logger.warning(...)` for the internal diagnostic (this is exactly the "this didn't work as expected" signal ops should see in logs, not a `print`), and separately returns a plain, user-facing `message` for the graph's actual result — the two are different audiences, so they use different channels.

Difference from Approach 1, and between Approaches 2/3: Approach 1 confirms the failure happens and reads the library's own `KeyError`. Approach 2 keeps the same fail-loudly behavior but gives it a message written for a human, and a specific `UnhandledRouteError` type, by validating inside `route()` before returning. Approach 3 changes the actual behavior — instead of failing, unexpected values route to a real, visible `"unhandled"` node — which is a legitimate choice for production code, as long as it's a deliberate mapping entry, not a missing one.

Which one should you actually write? During development, let it fail loudly — Approach 1's default `KeyError`, or better, Approach 2's specific `UnhandledRouteError`, so a routing bug gets caught immediately while you're building, not buried. Approach 3's fallback node is worth adding once a graph is heading to production and you've decided, on purpose, that an unexpected state value should degrade safely (log it, give a generic response) instead of taking down the whole run — but that's a decision to make deliberately, with a real node behind it, never a default you fall into by not handling a case.